A Practical Guide to Pangenome Graph Construction, Variant Calling, and Phenome-Wide Association in the HXB/BXH Rat Panel

Flavia Villani[1], … Pjotr Prins[2], Andrea Guarracino[1,*]

[1] Bioinnovation and Genome Sciences, The Translational Genomics Research Institute (TGen), Phoenix, AZ 85004, USA

[2] Department of Genetics, Genomics and Informatics, University of Tennessee Health Science Center, Memphis, TN 38163, USA

[*] Corresponding author: aguarracino@tgen.org

Abstract

A single reference genome cannot represent all genetic variation within a species, systematically biasing any analysis toward the reference allele and obscuring novel sequences and structural differences. Pangenomes reduce this bias by incorporating genomes from many individuals, thereby representing a broader spectrum of population diversity. A pangenome graph provides a unified model for encoding input genomes in a single data structure, where sequences are stored as nodes and their adjacency relationships as edges. Here we describe a practical methodology for constructing, validating, and leveraging a rat pangenome graph built from the HXB/BXH family of recombinant inbred (RI) rat strains, covering reference-free graph construction with PGGB, variant calling and validation, structural variant analysis, and phenome-wide association mapping. Applying this workflow, we identified novel variants absent from reference-based call sets and demonstrated their association with cardiometabolic and neurological phenotypes. We also discuss challenges in scaling pangenome analysis to larger cohorts and how implicit pangenome graphs may address these barriers, underscoring the value and future potential of pangenomic approaches for genotype–phenotype discovery.

Key Words: Pangenome, *Rattus norvegicus*, Pangenome graph, HXB/BXH strains, Recombinant inbred, Structural variants, PheWAS, Variant calling, Reference bias

1. Introduction

1.1. Limitations of single reference genomes

A single reference genome cannot faithfully represent the genetic variation within a species. Sequences present in some individuals but absent from the reference are invisible to any reference-anchored analysis, and regions that differ substantially from the reference are poorly characterized regardless of the analytical method used [1]. This representation gap introduces systematic reference bias: variant callers underperform in divergent regions, structural variants are missed or mischaracterized, and novel sequences carried by non-reference individuals can be lost [2]. These limitations have become increasingly apparent as whole-genome sequencing costs have decreased and researchers have begun characterizing genetic diversity across broader populations.

1.2. Pangenome graphs as a solution

A pangenome is defined as the complete set of genomic elements within a species or clade [3]. Moving from a single-reference framework to a pangenomic one means incorporating genetic variation from multiple individuals into a unified representation, rather than treating one genome as the standard against which all others are compared.

In a variation graph (also called a pangenome graph), DNA sequences are stored as nodes and the adjacency relationships between them as edges; technically, the graph is a bidirected sequence graph in which each node carries a DNA string and each edge connects a side (5′ or 3′) of one node to a side of another. A haplotype corresponds to a walk through this graph, and variant sites appear as bubbles—pairs of paths that diverge from and reconverge at shared anchor nodes. Each input genome is encoded as a walk traversing the graph [4]. Graphs are stored and exchanged in GFA (Graphical Fragment Assembly) format. This approach captures the full spectrum of population diversity, from single-nucleotide polymorphisms (SNPs) to large structural variants (SVs), and provides a data structure capable of simultaneously representing and analyzing hundreds of genomes at gigabase scale—a challenge that traditional reference-based methods cannot address [5]. Pangenome graphs have proven successful in representing human genetic variation through the Human Pangenome Reference Consortium (HPRC) and in addressing evolutionary questions across diverse taxa [4–6].

1.3. PGGB: Reference-free pangenome construction

Several methods exist for constructing pangenome graphs, but most are biased because they rely on reference-guided or phylogenetic-tree-guided approaches [7]. In these methods, the choice of reference or guide tree constrains the graph topology: variation is anchored to one genome's coordinate system, and structural differences not captured by that backbone can be missed or distorted. The PanGenome Graph Builder (PGGB) overcomes these limitations through a reference-free pipeline to construct unbiased pangenome graphs [8]. PGGB uses all-to-all whole-genome alignments, comparing every input sequence to every other input sequence without privileging any genome as a reference. All genomes are treated equivalently, without input order or phylogenetic dependencies, and each genome can serve as a potential reference for downstream analysis.

The PGGB pipeline integrates four core modules:

1. **WFMASH: All-to-all whole-genome alignment.** WFMASH performs all-to-all pairwise alignment of the input sequences [9]. It uses an extension of MashMap for rapid homology detection, finding segment-level similarities at a specified average nucleotide identity, and then computes base-level alignments using the bidirectional Wavefront Algorithm (BiWFA). By aligning every sequence directly against every other, WFMASH avoids reference or order bias: each sequence in the pangenome can serve as a potential reference.

2. **SEQWISH: Graph induction from alignments.** SEQWISH converts the set of input genomes together with their pairwise alignments into a variation graph [7]. In this graph, each input genome is fully embedded and can be extracted as a labeled path. SEQWISH also recovers transitive homology relationships—that is, if sequence A aligns to B and B aligns to C, the graph will reflect the relationship between A and C even if they were not directly aligned. SEQWISH scales to terabase-pair datasets through disk-backed processing.

3. **SMOOTHXG: Graph normalization.** The raw variation graph produced by SEQWISH contains spurious local complexity. SMOOTHXG resolves this by dividing the graph into blocks and realigning each block with Partial Order Alignment (POA), iterating to reduce edge effects. Multiple smoothing passes with different block-length targets can be chained for progressive refinement.

4. **GFAFFIX: Redundancy removal.** After smoothing, GFAFFIX collapses walk-preserving shared affixes—that is, redundant nodes that represent identical sequences shared across

paths—to produce a streamlined final graph. The output graph is then sorted using ODGI [10].

PGGB has been tested in the HPRC, where it contributed to the first draft human pangenome reference [5], and has been widely adopted for applications from microbial genomics [11] to crop improvement [12], evolutionary genomics [13], and mammalian reference genome projects [14–16]. The complete workflow described in this protocol is illustrated in Fig. 1.

An alternative approach to pangenome graph construction is Minigraph-Cactus (MC) [17], which uses a reference-guided strategy with minigraph for SV-scale backbone construction followed by Cactus for base-level alignment. PGGB is preferred when reference bias must be strictly avoided, when the evolutionary relationships among input genomes are unknown or complex, or when complete resolution of all variation (including within repetitive regions) is desired. MC may be more computationally efficient for very large numbers of genomes and produces graphs that are compatible with the vg Giraffe mapper for read alignment. The choice between the two depends on the biological question and the characteristics of the dataset (*see* Note 4).

1.4. HXB/BXH rat genetic reference population

The laboratory rat (*Rattus norvegicus*) is a widely used model organism for biomedical research. Wild *R. norvegicus* populations already harbor relatively limited genetic diversity compared with many other rodent species, and laboratory rat strains descend from a small number of wild-caught founders, creating an initial domestication bottleneck. Subsequent inbreeding to establish defined strains has further reduced diversity within each lineage [18]. Importantly, this two-stage bottleneck—first in the wild, then in the laboratory—means that a pangenome built from a modest number of inbred strains can capture a large fraction of the total genetic variation segregating among commonly used laboratory rats, making the pangenome approach particularly efficient in this species.

Genetic reference populations such as the HXB/BXH recombinant inbred (RI) strains serve as powerful tools for investigating genotype–phenotype relationships [19]. The HXB/BXH panel originated from crossing the Spontaneously Hypertensive Rat (SHR/OlaIpcv) and Brown Norway (BN-Lx/Cub) strains, creating 31 independent RI strains that segregate for alleles affecting blood pressure, metabolic traits and behavioral phenotypes relevant to human disease [20]. Because each RI strain is inbred, assemblies are effectively haploid, which simplifies pangenome construction: there is no need to phase diploid haplotypes, and each genome maps to a single path through the graph. Moreover, the parental strains of the HXB/BXH panel overlap with the founders of heterogeneous stock (HS) rat populations used for fine-mapping quantitative trait loci (QTLs); a pangenome that includes RI assemblies can therefore serve as a reference for imputation and variant discovery in those outbred cohorts as well.

The panel has been extensively phenotyped over more than 25 years, generating a vast dataset of quantitative molecular and physiological phenotypes available through GeneNetwork [21]. This combination of genetic stability, phenotypic depth, and relevance to human disease makes the HXB/BXH panel an ideal test case for demonstrating the utility of pangenome approaches in a model organism context.

1.5. Challenges and future directions

As pangenome projects scale to hundreds or thousands of genomes, two challenges become prominent. First, explicit graph construction remains computationally intensive: all-to-all alignment scales quadratically with the number of input genomes, building a graph from hundreds of

genomes requires days of computation and hundreds of gigabytes of memory, and adding a single new genome necessitates a full rebuild [8]. Second, the resulting graphs are large and complex data structures that resist interactive exploration; current visualization tools such as ODGI can produce static one- and two-dimensional layouts, but dynamic, on-the-fly visualization of pangenome graphs at genome scale remains an open problem.

Implicit pangenome graphs offer a complementary paradigm that addresses the scaling barrier [22]. Rather than collapsing all alignments into an explicit graph, the Implicit Pangenome Graph (IMPG) approach operates directly on pairwise alignments, indexing them with interval trees and traversing them to discover both direct and transitive homology relationships. Because each alignment file is indexed independently, incorporating a new genome requires only aligning it to a representative subset—an effort that scales linearly rather than quadratically with cohort size. This approach enables population-scale queries on commodity hardware and would allow researchers to extend the HXB/BXH pangenome incrementally as new strains are sequenced (*see* Note 14).

This protocol describes a complete computational workflow—no wet-lab procedures are covered—applied to the HXB/BXH panel: reference-free graph construction with PGGB, variant calling and validation, structural variant analysis, and phenome-wide association mapping linking pangenome-derived variants to the panel's extensive phenotype collection. The pre-built HXB/BXH pangenome graph and associated VCF files are available at github.com/Flavia95/HXB_rat_pangenome_manuscript.

2. Materials

2.1. Genome assemblies

1. Reference genome: mRatBN7.2 (rn7), available from NCBI [18].
2. De novo haploid genome assembly for each of the 31 HXB/BXH RI strains, generated from 10x Genomics Chromium Linked-Read whole-genome sequencing data at an average coverage depth of 109× using Supernova (version 2.1.1) [18]. Linked-Read technology bridges the gap between short-read and long-read approaches by providing long-range barcode information that links short reads originating from the same high-molecular-weight DNA molecule (*see* Note 1).
3. Gene annotations from Ensembl for the mRatBN7.2 assembly [23].
4. Phenotype data from GeneNetwork (genenetwork.org), comprising quantitative molecular and physiological phenotypes collected over more than 25 years across the HXB/BXH panel [21].

2.2. PGGB pipeline components

The PGGB [8] pipeline (available at https://github.com/pangenome/pggb) consists of four core modules whose algorithmic details are described in the Introduction (Section 1.3). Install PGGB using one of the following methods:

```
# Docker (recommended — includes all dependencies)
docker pull ghcr.io/pangenome/pggb:v0.7.0

# or Bioconda
```

```
conda create -n pggb -c bioconda -c conda-forge pggb=0.7.0
conda activate pggb
```

Singularity and Guix containers are also available. The individual module versions used in this protocol are:

1. **WFMASH** (v0.14.0): All-to-all whole-genome alignment. https://github.com/waveygang/wfmash [9].
2. **SEQWISH** (v0.7.11): Graph induction from pairwise alignments. https://github.com/ekg/seqwish [7].
3. **SMOOTHXG** (v0.8.0): Graph normalization via partial order alignment. https://github.com/pangenome/smoothxg.
4. **GFAFFIX** (v0.1.6): Walk-preserving redundancy removal. https://github.com/marschall-lab/GFAffix.

2.3. Downstream analysis tools

Install the downstream tools in a single Bioconda environment:

```
conda create -n pangenome-tools -c bioconda -c conda-forge \
    vg odgi samtools bcftools htslib rtg-tools \
    seqtk minimap2 snpeff snpsift survivor vcflib \
    fastix meryl mummer4 compleasm
conda activate pangenome-tools
```

Tools not available via Bioconda (PAV, SVIM-asm, Hall-lab pipeline, RepeatMasker) should be installed from their respective GitHub repositories listed below.

1. ODGI [10]: for computing graph statistics (node count, edge count, base content, path coverage) and for generating visualizations. ODGI provides both one-dimensional (1D) visualizations that show how paths align into the graph structure and two-dimensional (2D) visualizations that reveal graph topology. It can also produce pairwise distance matrices suitable for phylogenetic analysis.
2. vg deconstruct (from the vg toolkit) [1]: for extracting variants from the pangenome graph relative to a specified reference path by enumerating bubbles (snarls) in the graph.
3. BCFtools [24]: for VCF normalization, decomposition, filtering, and statistics.
4. vt (Variant Tool) [25]: for variant classification into simple (SNPs, MNPs, indels) and complex categories. A variant is classified as complex when its reference and alternate alleles overlap positionally but do not span the same range, indicating a compound event that cannot be decomposed into a single SNP or indel.
5. RTG Tools [26]: for precision/recall analysis of variant call sets using vcfeval.
6. SnpEff (v5.1) [27]: for functional annotation and effect prediction of variants on genes and proteins.
7. GEMMA [28] or GeneNetwork/BXDtools [29]: for kinship-corrected linear mixed model association analysis (PheWAS).
8. Assembly-based SV callers: PAV [30], SVIM-asm [31], and Hall-lab pipeline [32], used in combination with vg to produce a multi-method high-confidence SV call set.

9. SURVIVOR [33]: for merging structural variant calls across multiple callers.
10. vcfbub and vcfwave (from vcflib; https://github.com/vcflib/vcflib): for removing nested alleles and decomposing complex variants using the BiWFA algorithm, respectively.
11. RepeatMasker [34]: for masking low-complexity and repetitive regions.
12. Compleasm [35]: for BUSCO-based assessment of assembly completeness using the Mammalia ortholog gene set.
13. Minimap2 [36]: for assembly-to-reference alignment used by SV callers.

2.4. Hardware requirements

PGGB is computationally intensive. The HXB/BXH pangenome was built on a high-performance computing (HPC) cluster using machines equipped with AMD EPYC 7402P 24-core processors (48 threads), 256–378 GB of RAM, and 1 TB solid-state drive (SSD) storage. As a reference for runtime and memory requirements: building a pangenome graph of rat chromosome 12 from 32 haplotypes required approximately 29 GB of RAM, while human chromosome 6 from 90 haplotypes required approximately 1,183 minutes and 136 GB of RAM [8]. Docker and Singularity containers are available to simplify deployment. A cluster-scalable Nextflow implementation (nf-core/pangenome) is also available for distributing alignment jobs across multiple nodes [37].

3. Methods

All software listed in Section 2 should be installed before proceeding. The `filter_Ns.py` script for filtering N-rich SV calls is available at github.com/Flavia95/HXB_rat_pangenome_manuscript. Adjust the `-t` thread count in all commands below to match your available CPU cores.

3.1. Assembly quality assessment

Before constructing the pangenome, verify the quality and suitability of each de novo assembly. Assembly quality is evaluated using Compleasm [35] to assess the representation of universal single-copy orthologs expected across mammals using the BUSCO Mammalia gene set. Each assembly is benchmarked against the mRatBN7.2 reference genome [18].

1. Install Compleasm and download the Mammalia BUSCO gene set:

```
conda create -n compleasm -c conda-forge -c bioconda compleasm
conda activate compleasm
mkdir -p busco/databases
compleasm download -L busco/databases mammalia
```

2. Run Compleasm on each assembly and the reference genome:

```
mkdir -p busco/results
for ASSEMBLY in assemblies/*.fa.gz; do
    STRAIN=$(basename "$ASSEMBLY" .fa.gz)
    mkdir -p busco/results/${STRAIN}
    compleasm run \
        -a ${ASSEMBLY} \
        -o busco/results/${STRAIN} \
        -t 48 \
```

```
        -l mammalia \
        -L busco/databases
    mv busco/results/${STRAIN}/summary.txt \
        busco/results/${STRAIN}/summary.mammalia.${STRAIN}.txt
done
```

Adjust the `-t` parameter to match available CPU threads. On an HPC cluster, each strain can be submitted as an independent job. The majority of BUSCO orthologs should be complete and single-copy, which is indicative of high assembly quality.

3. Evaluate heterozygosity and genome characteristics. Use K-mer counting with Meryl [38] and GenomeScope [39] to estimate genome size, heterozygosity, and repeat content:

```
meryl count k=21 reads.fq.gz output sample.meryl
meryl histogram sample.meryl > sample.hist
```

Upload the histogram file to GenomeScope (https://qb.cshl.edu/genomescope/) to visualize the results.

4. Evaluate homozygosity of each strain by examining the fraction of heterozygous variant sites from standard reference-based variant calling (e.g., DeepVariant/GLnexus joint calling [40]). For inbred strains, close to 98% of variants should be homozygous. Flag any strain that deviates substantially (*see* Note 2).

5. Decontamination. Screen assemblies for mitochondrial, chloroplast, and other contamination. The NCBI Foreign Contamination Screen (FCS; https://github.com/ncbi/fcs) provides a standardized approach. For simplicity, remove contigs shorter than 100 kb:

```
samtools faidx assembly.fasta
awk '$2 > 100000 {print $1}' assembly.fasta.fai > keep.list
samtools faidx -r keep.list assembly.fasta > assembly.clean.fasta
samtools faidx assembly.clean.fasta
```

6. Identify telomeric repeats to assess assembly completeness at chromosome ends:

```
seqtk telo assembly.fasta > assembly.telo
```

3.2. PGGB pangenome construction

3.2.1. Input preparation

The PGGB pipeline outputs a pangenome graph in GFA format (version 1). Include the reference genome in the graph (*see* Note 3). Although PGGB is reference-free, including the reference is essential for downstream variant calling relative to an established coordinate system and for comparison with existing datasets.

1. Combine all haploid assemblies and the mRatBN7.2 reference into a single multi-FASTA file:

```
rm -f combined.fa
ls *.fa.gz | cut -f 1 -d '.' | uniq | while read f; do
    echo "$f"
```

```
    zcat "${f}".fa.gz >> combined.fa
done
```

2. Rename sequences according to the PanSN-spec (Pangenome Sequence Naming specification) convention: `sample#haplotype#contig`. For example: `SHR#1#chr12`. Because the assemblies are haploid, the haplotype field is set to 1 for all strains:

```
rm -f in.fa
ls *.fa.gz | cut -f 1 -d '.' | uniq | while read f; do
    echo "${f}"
    zcat "${f}".fa.gz | fastix -p "${f}#1#" >> in.fa
done
```

3. Compress and index:

```
bgzip -@ 4 in.fa
samtools faidx in.fa.gz
```

3.2.2. All-to-All Alignment (WFMASH)

Run PGGB, which internally invokes WFMASH as its first step. The command used for the HXB/BXH pangenome was:

```
pggb -i in.fa.gz \
    -p 98 -s 2000 -n 32 \
    -F 0.001 -k 79 -P asm5 -O 0.03 \
    -G 4001,4507 \
    -V rn7:# \
    -t 48 -T 48 \
    -o output_dir
```

The main parameters passed to WFMASH are the mapping identity minimum `-p` and the segment length `-s`. Key parameters (*see* Notes 5–6 for parameter selection guidance):

- `-p 98`: percent identity threshold for mapping. Set to 98% because the HXB/BXH strains are closely related inbred strains derived from two parental strains. For more divergent comparisons, lower values (e.g., 90–95%) are appropriate.
- `-s 2000`: segment length for mapping (in bp). This sets the minimum length of aligned segments.
- `-n 32`: number of haplotypes in the input (31 RI strains + 1 reference).
- `-F 0.001`: k-mer frequency filter threshold. Ignores the top 0.1% most frequent k-mers in the MinHash sketches, reducing spurious alignments in repetitive regions. For scaling the number of pairwise comparisons with large inputs, see the `-x` sparsification option (*see* Note 7).

WFMASH outputs alignments in PAF format. Each sequence is aligned directly against every other sequence, so that no single genome is privileged. The BiWFA algorithm computes base-level alignments from the segment-level mappings.

3.2.3. Graph induction (SEQWISH)

PGGB automatically invokes SEQWISH after alignment. SEQWISH reads the input genomes and the PAF alignments produced by WFMASH and induces a variation graph in GFA format. Graph induction often works better when very short exact matches are filtered out of the input alignments. These short matches typically occur in regions of low alignment quality, which are characteristic of areas with large indels and structural variations in the WFMASH alignments. A key parameter is:

- `-k 79`: minimum exact match length for graph induction. For the HXB/BXH pangenome, `-k 79` was used, appropriate for the low divergence between these closely related inbred strains (the default `-k 23` is suitable for divergences up to 5%). Setting `-k N` means that the graph can tolerate a local pairwise difference rate of no more than 1/N: indels represented by complex series of edit operations are opened into bubbles, and alignment regions with very low identity are ignored.

3.2.4. Graph normalization (SMOOTHXG)

The raw graph from SEQWISH contains spurious local complexity. SMOOTHXG refines the graph by running a partial order alignment (POA) across segments, called blocks. Key parameters:

- `-G 4001,4507`: target sequence length (in bp) for POA blocks. Two comma-separated values specify two successive smoothing passes with block targets of 4 kbp and 4.5 kbp respectively, providing progressive refinement. Higher values make sense for lower-diversity pangenomes.
- `-P asm5`: POA scoring parameters, using a minimap2-style preset tuned for assemblies with 0.1% divergence. This is appropriate for the closely related HXB/BXH strains.
- `-O 0.03`: POA block padding factor. Each block boundary is extended by 3% of the longest sequence in the block to improve alignment quality at block edges.
- `-T 48`: number of threads for POA steps.

3.2.5. Redundancy removal (GFAFFIX)

GFAFFIX collapses walk-preserving redundant nodes—nodes that share identical sequence prefixes or suffixes across all traversing paths. The final graph is sorted using ODGI. The output is a GFA file representing the complete pangenome variation graph.

3.2.6. Running PGGB chromosome by chromosome

For large genomes, it is practical to partition the input by chromosome and run PGGB independently on each chromosome (*see* Note 8). Use WFMASH to map assembly contigs against the reference to assign contigs to chromosomes, then run PGGB on each chromosome subset separately. This reduces memory requirements and allows parallel execution on a cluster.

Sequence partitioning. Map each assembly against the reference genome using WFMASH:

```
# a. Combine all assemblies into a single FASTA and index
cat assemblies/*.fasta.gz > combined.fasta.gz
samtools faidx combined.fasta.gz

# b. List sample assembly files (excluding the reference)
ls assemblies/*.fasta.gz | grep -v rn7 \
    | xargs -I{} basename {} | sort -V | uniq > haps.list
```

```
# c. Map each assembly against the reference
REF=assemblies/rn7.fasta.gz
mkdir -p alignments
for HAP in $(cat haps.list); do
    wfmash -t 48 -m -N -p 90 -s 20000 \
        ${REF} assemblies/${HAP} \
        > alignments/${HAP}.vs.ref.paf
done
```

The `-m` flag requests mapping-only output (no base-level alignment), `-N` prevents query splitting (maps each sequence in one piece), `-p 90` sets a permissive identity threshold to capture divergent contigs, and `-s 20000` sets a 20 kbp segment length appropriate for chromosome-scale assignment.

Subset by chromosome. Extract contig names per chromosome and build chromosome-specific FASTAs:

```
# d. Extract contig names assigned to each chromosome
mkdir -p parts
for CHR in $(seq 1 20) X Y; do
    awk -v chr="chr${CHR}" '$6 ~ chr"$"' \
        alignments/*.vs.ref.paf \
        | cut -f 1 | sort -V \
        > parts/chr${CHR}.contigs
done

# e. Build per-chromosome FASTAs from mapped contigs
for CHR in $(seq 1 20) X Y; do
    xargs samtools faidx combined.fasta.gz \
        < parts/chr${CHR}.contigs \
        > parts/chr${CHR}.pan.fa
done

# f. Add the reference chromosome and compress
# (chrM is excluded: the mitochondrial genome is small
#  and has a distinct evolutionary history)
for CHR in $(seq 1 20) X Y; do
    samtools faidx ${REF} rn7#1#chr${CHR} \
        > parts/rn7.chr${CHR}.fa
    cat parts/rn7.chr${CHR}.fa parts/chr${CHR}.pan.fa \
        > parts/chr${CHR}.pan+ref.fa
    bgzip parts/chr${CHR}.pan+ref.fa
    samtools faidx parts/chr${CHR}.pan+ref.fa.gz
done
```

The output is a set of bgzip-compressed, indexed per-chromosome FASTAs (`chr${CHR}.pan+ref.fa.gz`), each containing all strain contigs mapping to that chromosome plus the corresponding reference sequence.

Run PGGB per chromosome:

```
for CHR in $(seq 1 20) X Y; do
    pggb \
```

```
        -i parts/chr${CHR}.pan+ref.fa.gz \
        -p 98 -s 2000 -n 32 \
        -F 0.001 -k 79 -P asm5 -O 0.03 \
        -G 4001,4507 \
        -V rn7:# \
        -t 48 -T 48 \
        -o output/chr${CHR}.pan+ref
done
```

Each chromosome can be submitted as an independent job on an HPC cluster. Using a local scratch disk for intermediate files and moving results to persistent storage upon completion is recommended.

3.3. Graph quality assessment

After construction, evaluate graph quality using ODGI and diagnostic visualizations.

1. Graph statistics. Compute total graph length, node count, edge count, path count, and non-reference sequence with odgi stats:

```
odgi stats -i graph.og -S
```

For MultiQC-compatible output, add the `-m -sgdl` flags:

```
odgi stats -i graph.og -m -sgdl
```

2. 1D visualization. Generate a linearized view of all paths against the pangenome sequence with odgi viz:

```
odgi viz -i graph.og -o graph.1D.png -x 500
```

Additional options: `-z` colors bars by node strandedness (black = forward, red = reverse); `-bm` colors bars by mean coverage depth (black = low, green = high).

3. 2D visualization. First compute a 2D layout, then render it as an image:

```
odgi layout -i graph.og -o graph.og.lay
odgi draw -i graph.og -c graph.og.lay -p graph.2D.png
```

4. Subgraph extraction. Extract and inspect specific regions of interest:

```
# By node ID (with context of 1 step)
odgi extract -i graph.og -n 23 -c 1 -o subgraph.og

# By reference coordinates (e.g., a 1 Mbp window on chr12)
odgi extract -i graph.og -r rn7#1#chr12:1000000-2000000 \
    -o subgraph.og
```

The 1D visualization produces a horizontal image in which each row represents a path (genome) through the graph; colored segments show how each path traverses graph nodes, with gaps indicating sequence absent from that path. The 2D layout reveals the topological structure of the graph: linear stretches appear as straight lines, while bubbles (variant sites) and complex

rearrangements produce visible branching patterns. Both views are useful for quality control—for example, a large inversion will appear as a red (reverse-strand) segment in the 1D view and as a loop in the 2D view. For further discussion of visualization challenges and limitations, *see* Note 13.

3.4. Read mapping to the pangenome

Once the pangenome graph is constructed, it can serve as a reference for aligning short reads using vg Giraffe [17], which maps reads to a graph index:

1. Build graph indexes. Convert the GFA to the formats required by vg Giraffe:

```
# Convert GFA to vg format and compute snarls
vg autoindex --workflow giraffe \
    -g graph.gfa \
    -p graph_index \
    -t 48
```

The `autoindex` command creates the distance index, minimizer index, and GBWT haplotype index needed by Giraffe.

2. Map reads. Align paired-end reads against the pangenome graph:

```
vg giraffe \
    -Z graph_index.giraffe.gbz \
    -m graph_index.min \
    -d graph_index.dist \
    -f reads_1.fq.gz -f reads_2.fq.gz \
    -t 48 \
    -o gaf \
    > aligned.gaf
```

The output is in GAF (Graph Alignment Format). To convert to BAM for use with standard tools, surject the alignments onto the reference path:

```
vg surject -x graph_index.giraffe.gbz \
    -t 48 -b -N sample_name -R "sample_name" \
    aligned.gaf > aligned.bam
samtools sort -@ 48 aligned.bam -o aligned.sorted.bam
samtools index aligned.sorted.bam
```

Note that vg Giraffe requires a graph built by Minigraph-Cactus or autoindexed from GFA. When using a PGGB graph, the `vg autoindex` step handles the necessary conversions. For large-scale read mapping projects, Minigraph-Cactus graphs may offer better Giraffe compatibility (*see* Note 4).

3.5. Variant calling from the pangenome

Variants are extracted from the pangenome graph relative to the reference path using vg deconstruct, which identifies bubbles—pairs of divergent paths between shared anchors—and decomposes them into VCF records.

- `-V rn7:#`: specifies the reference prefix for VCF output. Variants are called relative to paths matching this prefix. vg deconstruct is invoked automatically by PGGB when the `-V` flag is set.

Normalize and decompose the resulting VCF using BCFtools:

```
bcftools norm -m -both -f rn7.fa output.vcf.gz | \
    bcftools view -e 'ALT="*"' | \
    bcftools sort -Oz -o normalized.vcf.gz
```

Perform reference-based variant calling using DeepVariant/GLnexus [40] on the same sequencing data as a benchmark call set for validation (Section 3.5). Annotate variants using SnpEff [27] for functional consequence prediction:

```
snpEff mRatBN7.2 input.vcf > annotated.vcf
```

3.6. Validation strategies

3.6.1. Cross-validation with reference-based Joint Calling

Process both the pangenome-derived and reference-based call sets to remove missing data, N-allele sites, homozygous reference genotypes, and variants >50 bp. Compare call sets using RTG Tools vcfeval with the `--squash-ploidy` option (since the inbred strains are essentially isogenic, haploid pangenome calls are treated as homozygous diploid for comparison):

```
# Create an SDF from the reference FASTA
rtg format -o ref.sdf reference.fa

# Run vcfeval to compute precision, recall, and F1-score
rtg vcfeval \
    -t ref.sdf \
    -b truth/sample.joint_calling.snps.vcf.gz \
    -c pangenome/sample.pggb.snps.vcf.gz \
    --squash-ploidy \
    -e easy_regions.bed \
    -T 48 \
    -o vcfeval_output/snps
```

Here `-b` specifies the baseline (truth) VCF from reference-based joint calling, `-c` is the pangenome-derived call set to evaluate, `-e` restricts evaluation to confidently callable regions (e.g., RepeatMasker-derived easy regions excluding low-complexity repeats), and `-T` sets the number of threads. The output contains precision, recall, F1-score, and partitioned true-positive, false-positive, and false-negative VCFs (*see* Note 9).

3.6.2. Cross-validation with MUMMER4

An independent validation approach uses NUCMER (part of MUMMER4 [41]) to generate pairwise alignments between individual genome sequences and call variants from these alignments:

```
# Align sample assembly against the reference
nucmer --maxmatch -l 100 -c 500 \
    reference.fa sample.fa -p sample
```

```
# Filter alignments and call variants
delta-filter -1 sample.delta > sample.filtered.delta
show-snps -Clr sample.filtered.delta > sample.snps
```

The NUCMER-derived variants are converted to VCF format and compared against the pangenome-derived call set using RTG Tools vcfeval as described above. F1-scores exceeding 90% have been obtained across diverse genomic contexts [8].

3.7. Structural variant analysis

Structural variants (≥50 bp) are called from the pangenome using a multi-method approach to maximize sensitivity and specificity. For the full pipeline and detailed analysis, see [14].

1. Assembly-based SV calling. Align each sample assembly against the reference with minimap2 [36] and call SVs using three independent methods:

```
REF=reference.fa
for SAMPLE in $(cat samples.list); do
    # Align assembly to reference
    minimap2 -ax asm5 -L --cs -t 48 ${REF} \
        assemblies/${SAMPLE}.fa \
        | samtools sort -m 8G -@ 48 -o ${SAMPLE}.bam -
    samtools index ${SAMPLE}.bam

    # Method 1: SVIM-asm (haploid mode)
    svim-asm haploid svim_out/ ${SAMPLE}.bam ${REF} \
        --sample ${SAMPLE} --query_names \
        --interspersed_duplications_as_insertions \
        --min_sv_size 50

    # Method 2: PAV
    # Requires config.json pointing to reference and
    # assemblies.tsv listing sample paths (see PAV docs)

    # Method 3: Hall-lab pipeline
    # Uses paftools.js call for variant detection from
    # the minimap2 alignment, followed by VCF conversion
done
```

For each caller, normalize and filter the output to retain only SVs (≥50 bp):

```
bcftools norm -m -any -f ${REF} ${SAMPLE}.raw.vcf.gz -cs \
    | bcftools norm -d exact \
    | bcftools view -i 'STRLEN(REF)>=50 | STRLEN(ALT)>=50' \
    | bcftools sort | bgzip -c > ${SAMPLE}.caller.vcf.gz
tabix ${SAMPLE}.caller.vcf.gz
```

2. Graph-based SV calling from PGGB. Extract per-sample SVs from the pangenome VCF produced by vg deconstruct. Use vcfbub to remove nested alleles and vcfwave to decompose complex variants:

```
# Decompose complex alleles from the pangenome VCF
vcfbub -l 0 -a 100000 --input pangenome.vcf.gz \
    | vcfwave -t 48 -I 1000 \
    | bgzip -c > pangenome.wave.vcf.gz

# Extract per-sample SVs
for SAMPLE in $(cat samples.list); do
    bcftools annotate -x INFO pangenome.wave.vcf.gz \
        | bcftools view -a -s ${SAMPLE} -Ou \
        | bcftools norm -f ${REF} -c s -m - -Ou \
        | bcftools view -e 'GT="ref" | GT~"\."' \
            -f 'PASS,.' -Ou \
        | bcftools sort -Ou \
        | bcftools norm -d exact \
        | bcftools view \
            -i 'STRLEN(REF)>=50 | STRLEN(ALT)>=50' \
        | bgzip -c > ${SAMPLE}.pggb.vcf.gz
done
```

3. Multi-method merging. Merge SV calls across all four methods per sample using SURVIVOR [33], requiring agreement from at least two callers on type and strand within a 1,000 bp breakpoint distance:

```
for SAMPLE in $(cat samples.list); do
    # List per-caller VCFs for this sample
    printf '%s\n' ${SAMPLE}.hall-lab.vcf.gz \
        ${SAMPLE}.pggb.vcf.gz ${SAMPLE}.svim-asm.vcf.gz \
        ${SAMPLE}.pav.vcf.gz > ${SAMPLE}.file_list

    # Merge: 1000bp max distance, >=2 callers,
    # agree on type and strand, min SV size 50bp
    SURVIVOR merge ${SAMPLE}.file_list \
        1000 2 1 1 0 50 ${SAMPLE}.merged.vcf

    # Filter variants with >10% Ns in REF/ALT
    filter_Ns.py 10 < ${SAMPLE}.merged.vcf \
        | bcftools sort \
        | bgzip -c > ${SAMPLE}.merged.filtered.vcf.gz
    tabix ${SAMPLE}.merged.filtered.vcf.gz
done
```

4. Cross-sample merging. Merge the per-sample filtered call sets into a single cohort VCF:

```
ls *.merged.filtered.vcf.gz | while read f; do
    SAMPLE=$(basename $f .merged.filtered.vcf.gz)
    bcftools view \
        -s $(bcftools query -l $f | head -1) $f \
        > ${SAMPLE}.vcf
done

ls *.merged.filtered.vcf.gz > sample_files
SURVIVOR merge sample_files 1000 2 1 1 0 50 \
    all_samples.merged.vcf
```

```
bcftools sort all_samples.merged.vcf \
    | bgzip -c > all_samples.merged.vcf.gz
tabix all_samples.merged.vcf.gz
```

Note that PGGB's graph-based method typically reports fewer SVs than assembly-based methods (*see* Note 10).

5. Complex SV inspection. For complex SVs, extract the local subgraph using ODGI and visualize with Bandage or ODGI to reveal all realized haplotypes. For example, a complex insertion in the *Cd209c* gene was resolved into multiple variable blocks forming distinct haplotypes across the RI panel (*see* Note 11).

6. Annotate SV content with RepeatMasker [34] to identify retrotransposon content (LINEs, SINEs) within structural variants.

3.8. Phenome-Wide Association Study (PheWAS)

The pangenome enables discovery of novel variants that can be immediately tested for phenotypic associations using the extensive HXB/BXH phenotype database.

1. Prepare the genotype file. Extract genotypes for validated pangenome-only variants and convert to a BXDtools-compatible format:

```
# Extract genotype calls from validated variants
bcftools query -f '%CHROM\t%POS\t[ %GT]\n' \
    validated_variants.vcf.gz \
    -o validated.gt.txt
```

Convert to a genotype matrix in R:

```
# Read genotype calls and sample names
vcf <- read.table("validated.gt.txt")
samples <- read.table("samples.txt", header = FALSE)
colnames(vcf) <- c("Chr", "POS", samples$V1)

# Create locus identifiers (e.g., chr12_4347739)
vcf$Locus <- paste0(vcf$Chr, "_", vcf$POS)
vcf <- vcf[!duplicated(vcf$Locus), ]

# Recode: 0 (ref) -> "B", 1 (alt) -> "A", missing -> "U"
vcf[vcf == "0"] <- "B"
vcf[vcf == "1"] <- "A"
vcf[vcf == "."] <- "U"

# Add required columns and format for BXDtools
vcf$cM <- "."
vcf$Mb <- vcf$POS / 1e6
vcf$Chr <- gsub("chr", "", vcf$Chr)
write.table(vcf, "HXB.geno", row.names = FALSE,
    sep = "\t", quote = FALSE)
```

2. Download phenotype data and run PheWAS. Retrieve trait metadata and sample data from GeneNetwork via the API, then run association analysis using an adaptation of BXDtools [29]:

```
library(BXDtools)

# GeneNetwork API endpoints:
# genenetwork.org/api/v_pre1/traits/HXBBXHPublish.csv
# genenetwork.org/api/v_pre1/sample_data/HXBBXHPublish.csv

# Load genotypes and recode to numeric
bxd.geno <- read.table("HXB.geno", sep = "\t",
    header = TRUE, row.names = 1)
bxd.genotypes <- recode.BXD.genotypes(
    only.BXD.genotypes(bxd.geno))

# Load phenotypes, match samples to genotype file
bxd.pheno <- download.BXD.phenotypes()
bxd.genotypes <- bxd.genotypes[,
    colnames(bxd.genotypes) %in% colnames(bxd.pheno)]

# Build phenotype matrix, filter, group phenosomes
bxd.phenotypes <- as.phenotype.matrix(
    bxd.genotypes, bxd.pheno)
bxd.phenosomes <- only.phenosomes(
    bxd.phenotypes, minimum = 1)

# Run PheWAS for a validated marker
marker <- "chr12_4347739"
scores <- do.BXD.phewas(bxd.genotypes,
    bxd.phenosomes, marker = marker, LRS = TRUE)

# Plot with LRS > 16 significance threshold
pdf("phewas_result.pdf", width = 7, height = 7)
plot.phewas(scores, bxd.phenosomes, do.sort = TRUE,
    main = paste0("HXB PheWAS at ", marker),
    significance = 16)
dev.off()
```

The `do.BXD.phewas` function computes Spearman correlations between genotype and each phenotype, with Bonferroni correction for multiple testing. Significant associations (LRS $>$ 16) are reported. Adapted scripts are available at https://github.com/Flavia95/HXB_rat_pangenome_manuscript/blob/main/workflows/3_PheWAS.md.

3. Confirm associations. Validate significant PheWAS hits using a linear mixed model corrected for kinship, as implemented in GeneNetwork or GEMMA [28], to account for the RI population structure (*see* Note 12).

4. Functional annotation. Cross-reference significant associations with previously mapped QTLs using the Rat Genome Database (RGD; rgd.mcw.edu) [42] and the Ensembl Genome Browser [23] selecting the mRatBN7.2 reference genome [18].

Expected results. Applying this workflow to the HXB/BXH panel, the following associations were identified [14]: (a) A variant (chr12_4347739) within a long non-coding RNA gene was

associated with blood glucose concentration and located on the same chromosome as a previously mapped QTL controlling insulin/glucose ratio (Insglur6, LOD 18.97). (b) An intronic variant (chr12_18797475) within a locus similar to the paired immunoglobulin-like type 2 receptor was associated with both blood insulin concentration and hippocampal chromogranin A (CGA) expression. Additionally, validated SVs were found in disease-relevant genes including *Lmtk2* (implicated in neurodegeneration, including Alzheimer's disease) and *Mcemp1* (a critical factor in allergic and inflammatory lung diseases).

4. Notes

1. **Linked-Read technology considerations.** 10x Genomics Chromium Linked-Reads are not ideal for de novo assembly compared to long-read technologies (PacBio HiFi, Oxford Nanopore). Linked-Read assemblies tend to have more unresolved regions (stretches of Ns) than long-read assemblies, which leads to an overestimation of non-reference sequence when Ns are included in the counting. In the HXB/BXH pangenome, the non-reference content rose from 0.18 Gb (excluding Ns) to 3.9 Gb (including Ns). Despite this limitation, the pangenome still captured substantial genuine novel variation. As long-read sequencing costs continue to decrease, combining Linked-Read data with long-read data offers the prospect of higher-quality assemblies and more complete pangenomes.

2. **Handling heterozygous strains.** Among the 31 HXB/BXH strains, BXH2 showed 7.7% heterozygous variants, likely due to a recent breeding error [18]. In principle, such a strain can still be included in the pangenome as a haploid assembly if the assembler selected one haplotype consistently. However, results for this strain should be interpreted with caution, as the assembly may switch between haplotypes in some regions.

3. **Reference genome inclusion rationale.** Although PGGB is reference-free, including the standard reference genome (mRatBN7.2) in the graph is essential for practical reasons: it provides a coordinate system for variant calling (via the `-V` flag), enables comparison with existing reference-based datasets, and allows annotation with established gene models. The reference is treated as just another genome in the graph and does not receive any special treatment during construction.

4. **Choosing between PGGB and Minigraph-Cactus.** Both tools can produce high-quality pangenome graphs, but they suit different scenarios. Prefer PGGB when: (i) you need a strictly reference-free graph (e.g., no single genome is a natural reference, or you want to avoid any reference bias); (ii) the input set is moderate in size (tens to low hundreds of haplotypes); (iii) full base-level resolution of all variation, including within repetitive regions, is important. Prefer Minigraph-Cactus when: (i) a reliable reference genome exists and a reference-anchored graph is acceptable; (ii) the input set is very large (hundreds to thousands of haplotypes) and computational efficiency is a priority; (iii) downstream read mapping with vg Giraffe is planned. For the HXB/BXH panel (32 haplotypes, closely related inbred strains), PGGB was the natural choice because reference-free construction avoids privileging either parental genome.

5. **Percent identity parameter selection.** The `-p` parameter sets the minimum percent identity for WFMASH mappings. For closely related inbred strains derived from the same species (such as HXB/BXH rats or BXD mice), values of 95–98% are appropriate. For interspecies comparisons or highly divergent populations, lower values (85–90%) may be needed. Setting this value too low will increase runtime and may introduce spurious alignments; setting it too high will miss genuinely divergent regions.

6. **Segment length parameter.** The `-s` parameter controls the minimum length of mapping segments. Shorter values increase sensitivity but also increase runtime and may capture more spurious short alignments. For closely related mammalian genomes, values between 1,000 and 10,000 bp typically work well.

7. **Sparsification and k-mer filtering for large genome numbers.** All-to-all alignment scales quadratically with the number of input genomes. Two PGGB parameters help manage this. The `-F` parameter sets a k-mer frequency filter threshold: setting `-F 0.001` ignores the top 0.1% most frequent k-mers in MinHash sketches, reducing spurious alignments in repetitive regions. For scaling the number of pairwise comparisons themselves, the `-x` parameter activates Erdős–Rényi random sparsification, which randomly subsamples the set of pairwise comparisons. Because SEQWISH recovers transitive homology relationships (if A aligns to B and B to C, then A–C is inferred), the loss of direct alignments is mitigated. With sparsification, a 10× increase in input genomes requires approximately 20× runtime increase rather than 100×. For 32 genomes, the benefit is moderate; for hundreds of genomes, it becomes essential.

8. **Chromosome-by-chromosome construction.** For mammalian-scale genomes, constructing the pangenome one chromosome at a time is practical and recommended. This approach reduces peak memory requirements, enables parallel execution across cluster nodes, and simplifies debugging. The resulting per-chromosome GFA and VCF files can be concatenated for genome-wide analyses.

9. **Variant calling precision in repetitive regions.** Genome-wide accuracy of pangenome-derived variants compared to joint calling was approximately 90%. Exceptions included chromosomes Y and 16, where assembly complexity (chromosome Y) and an abundance of complex variants (chromosome 16) reduced concordance. Within repetitive regions, Sanger validation showed that approximately 90% of unconfirmed variants were located within repeats, suggesting that variant calling in these regions remains challenging even with pangenomic approaches.

10. **Why does PGGB report fewer SVs than traditional methods?** Graph-based SV calling from PGGB typically reports fewer SVs than assembly-based methods such as PAV or SVIM-asm. This is not a deficiency. PGGB aligns through variable regions, resolving what appears as a single large SV in a linear representation into a series of smaller variants (SNPs, MNPs, small indels) that more accurately describe the actual sequence differences. The multi-method approach (retaining SVs supported by ≥2 callers) balances sensitivity and specificity.

11. **Complex SV interpretation: the Cd209c example.** The pangenome graph fully resolved a complex 82 bp insertion in the *Cd209c* gene into 7 variable blocks that combined into 6 distinct haplotypes across the RI panel, 4 of which were observed fewer than two times [14, 15, 43]. This level of resolution is impossible with linear reference-based approaches, which would report a single insertion event. The insertion contained an Lx8b_3end LINE retrotransposon. The human ortholog of *Cd209c* is implicated in multiple infectious diseases. Such complex variants in immune-related genes, enriched for transposable elements, may facilitate rapid adaptive evolution of immune responses.

12. **PheWAS significance thresholds for RI strains.** The appropriate significance threshold for PheWAS in RI populations depends on the number of strains and phenotypes tested. With 30 strains and 60 phenotype classes, we used Bonferroni-corrected Spearman

correlations (adjusted $p < 0.05$) for initial screening, followed by confirmation with a linear mixed model correcting for kinship (LRS > 16). The kinship correction is essential because RI strains share varying degrees of relatedness, and failure to account for this structure inflates false positives.

13. **Visualization challenges and alternatives.** Current pangenome visualization tools, including ODGI, produce static images. For small subgraphs, tools such as Bandage and GfaViz offer interactive exploration, but no tool yet supports real-time dynamic navigation of chromosome-scale or genome-wide pangenome graphs. This remains an open challenge: the data structures are large, the topology is complex, and meaningful layout at multiple zoom levels is computationally expensive. For now, the practical strategy is to generate genome-wide overview images (1D and 2D) for quality control and then extract subgraphs for detailed interactive inspection of regions of interest.

14. **Scaling pangenome analysis with implicit pangenome graphs.** Explicit pangenome graph construction remains computationally intensive: building a graph from hundreds of genomes requires days of computation and hundreds of gigabytes of memory, and adding a single new genome necessitates a full rebuild [8]. Moreover, collapsing alignments into graph nodes discards alignment-level information such as pairwise identity and structural context. Implicit pangenome graphs offer a complementary paradigm that addresses these scaling barriers [22] (note: at the time of writing, the IMPG manuscript is under review). The Implicit Pangenome Graph (IMPG) toolkit operates directly on pairwise alignments, indexing them with interval trees and traversing them to discover both direct and transitive relationships between sequences. Compact alignment representations, compressed genome archives, and incremental per-file indexing together enable population-scale analysis on commodity hardware—for example, querying a 6 Mbp region across 466 human haplotypes in 1–2 seconds with peak memory under 100 MB. Because each alignment file is indexed independently, incorporating a new genome requires only aligning it to a representative subset, an effort that scales linearly rather than quadratically with cohort size. In the context of the HXB/BXH panel, an implicit graph approach would allow researchers to add newly sequenced RI strains or heterogeneous stock haplotypes without rebuilding the entire pangenome, and to rapidly extract multi-haplotype sequence sets for any region of interest.

Competing Interests

The authors declare that they have no competing interests.

Acknowledgments

This work was supported in part by the National Institutes of Health (NIH). We thank the members of the HPRC and the pangenome community for discussions and tool development. The HXB/BXH RI strains were maintained by the Institute of Physiology, Czech Academy of Sciences.

Bibliography

1. Garrison E, Sirén J, Novak AM (2018) Variation graph toolkit improves read mapping by representing genetic variation in the reference. Nature Biotechnology 36:875–879. https://doi.org/10.1038/nbt.4227

2. Paten B, Novak AM, Eizenga JM (2017) Genome graphs and the evolution of genome inference. Genome Research 27:665–676. https://doi.org/10.1101/gr.214155.116

3. Eizenga JM, Novak AM, Sibbesen JA (2020) Pangenome Graphs. Annual Review of Genomics and Human Genetics 21:139–162. https://doi.org/10.1146/annurev-genom-120219-080406

4. Secomandi S, Gallo GR, Rossi R (2025) Pangenome graphs and their applications in biodiversity genomics. Nature Genetics 57:13–26. https://doi.org/10.1038/s41588-024-02029-6

5. Liao W-W, Asri M, Ebler J (2023) A draft human pangenome reference. Nature 617:312–324. https://doi.org/10.1038/s41586-023-05896-x

6. Wang T, Antonacci-Fulton L, Howe K (2022) The Human Pangenome Project: a global resource to map genomic diversity. Nature 604:437–446. https://doi.org/10.1038/s41586-022-04601-8

7. Garrison E, Guarracino A (2023) Unbiased pangenome graphs. Bioinformatics 39:. https://doi.org/10.1093/bioinformatics/btac743

8. Garrison E, Guarracino A, Heumos S (2024) Building pangenome graphs. Nature Methods 21:2008–2012. https://doi.org/10.1038/s41592-024-02430-3

9. Guarracino A, Mwaniki N, Marco-Sola S (2021) wfmash: whole-chromosome pairwise alignment using the hierarchical wavefront algorithm. https://github.com/waveygang/wfmash

10. Guarracino A, Heumos S, Nahnsen S (2022) ODGI: understanding pangenome graphs. Bioinformatics 38:3319–3326. https://doi.org/10.1093/bioinformatics/btac308

11. Shoer S, Reicher L, Zhao C (2024) Pangenomes of human gut microbiota uncover links between genetic diversity and stress response. Cell Host & Microbe 32:1744–1757.e2. https://doi.org/10.1016/j.chom.2024.08.017

12. Schreiber M, Jayakodi M, Stein N (2024) Plant pangenomes for crop improvement, biodiversity and evolution. Nature Reviews Genetics 25:563–577. https://doi.org/10.1038/s41576-024-00691-4

13. Bolognini D, Halgren A, Lou RN (2024) Recurrent evolution and selection shape structural diversity at the amylase locus. Nature 634:617–625. https://doi.org/10.1038/s41586-024-07911-1

14. Villani F, Guarracino A, Ward RR (2025) Pangenome reconstruction in rats enhances genotype-phenotype mapping and variant discovery. iScience 28:111835. https://doi.org/10.1016/j.isci.2025.111835

15. Villani F (2025) Deciphering Genotype-Phenotype Connections: Leveraging Pangenomes and Comprehensive Genome Variation

16. Helmy M, Li JU, Yan XF (2026) High-quality mouse reference genomes reveal the structural complexity of the murine protein-coding landscape. Cell Genomics 6:101074. https://doi.org/10.1016/j.xgen.2025.101074

17. Hickey G, Monlong J, Ebler J (2024) Pangenome graph construction from genome alignments with Minigraph-Cactus. Nature Biotechnology 42:663–673. https://doi.org/10.1038/s41587-023-01793-w

18. Jong TV de, Pan Y, Rastas P (2024) A revamped rat reference genome improves the discovery of genetic diversity in laboratory rats. Cell Genomics 4:100527. https://doi.org/10.1016/j.xgen.2024.100527

19. Pravenec M, Klir P, Kren V (1989) An analysis of spontaneous hypertension in spontaneously hypertensive rats by means of new recombinant inbred strains. Journal of Hypertension 7:217–221

20. Printz MP, Jirout M, Jaworski R (2003) Invited Review: HXB/BXH rat recombinant inbred strain platform: a newly enhanced tool for cardiovascular, behavioral, and developmental genetics and genomics. Journal of Applied Physiology 94:2510–2522. https://doi.org/10.1152/japplphysiol.00064.2003

21. Mulligan MK, Mozhui K, Prins P (2017) GeneNetwork: A Toolbox for Systems Genetics. Methods in Molecular Biology 1488:75–120. https://doi.org/10.1007/978-1-4939-6427-7_4

22. Guarracino A, Kille B, Garrison E (2025) Implicit pangenome graphs for accessible pangenomics. Bioinformatics

23. Martin FJ, Amode MR, Aneja A (2023) Ensembl 2023. Nucleic Acids Research 51:D933–D941. https://doi.org/10.1093/nar/gkac958

24. Danecek P, Bonfield JK, Liddle J (2021) Twelve years of SAMtools and BCFtools. GigaScience 10:. https://doi.org/10.1093/gigascience/giab008

25. Tan A, Abecasis GR, Kang HM (2015) Unified representation of genetic variants. Bioinformatics 31:2202–2204. https://doi.org/10.1093/bioinformatics/btv112

26. Cleary JG, Braithwaite R, Gaastra K (2015) Comparing Variant Call Files for Performance Benchmarking of Next-Generation Sequencing Variant Calling Pipelines. bioRxiv. https://doi.org/10.1101/023754

27. Cingolani P, Platts A, Wang LL (2012) A program for annotating and predicting the effects of single nucleotide polymorphisms, SnpEff: SNPs in the genome of Drosophila melanogaster strain w 1118 ; iso-2; iso-3. Fly 6:80–92. https://doi.org/10.4161/fly.19695

28. Zhou X, Stephens M (2012) Genome-wide efficient mixed-model analysis for association studies. Nature Genetics 44:821–824. https://doi.org/10.1038/ng.2310

29. Arends D BXDtools: genetic tools for working with BXD and other model organism populations. https://github.com/DannyArends/BXDtools

30. Ebert P, Audano PA, Zhu Q (2021) Haplotype-resolved diverse human genomes and integrated analysis of structural variation. Science 372:. https://doi.org/10.1126/science.abf7117

31. Heller D, Vingron M (2020) SVIM-asm: structural variant detection from haploid and diploid genome assemblies. Bioinformatics 36:5519–5521. https://doi.org/10.1093/bioinformatics/btaa1034

32. Hall Lab Assembly validation pipeline. https://github.com/hall-lab/assembly_validation

33. Jeffares DC, Jolly C, Hoti M (2017) Transient structural variations have strong effects on quantitative traits and reproductive isolation in fission yeast. Nature Communications 8:. https://doi.org/10.1038/ncomms14061

34. Tarailo-Graovac M, Chen N (2009) Using RepeatMasker to Identify Repetitive Elements in Genomic Sequences. Current Protocols in Bioinformatics 25:. https://doi.org/10.1002/0471250953.bi0410s25

35. Huang N, Li H (2023) compleasm: a faster and more accurate reimplementation of BUSCO. Bioinformatics 39:. https://doi.org/10.1093/bioinformatics/btad595

36. Li H (2018) Minimap2: pairwise alignment for nucleotide sequences. Bioinformatics 34:3094–3100. https://doi.org/10.1093/bioinformatics/bty191

37. Heumos S, Heuer ML, Hanssen F (2024) Cluster-efficient pangenome graph construction with nf-core/pangenome. Bioinformatics 40:. https://doi.org/10.1093/bioinformatics/btae609

38. Rhie A, Walenz BP, Koren S (2020) Merqury: reference-free quality, completeness, and phasing assessment for genome assemblies. Genome Biology 21:. https://doi.org/10.1186/s13059-020-02134-9

39. Vurture GW, Sedlazeck FJ, Nattestad M (2017) GenomeScope: fast reference-free genome profiling from short reads. Bioinformatics 33:2202–2204. https://doi.org/10.1093/bioinformatics/btx153

40. Yun T, Li H, Chang P-C (2021) Accurate, scalable cohort variant calls using DeepVariant and GLnexus. Bioinformatics 36:5582–5589. https://doi.org/10.1093/bioinformatics/btaa1081

41. Marçais G, Delcher AL, Phillippy AM (2018) MUMmer4: A fast and versatile genome alignment system. PLOS Computational Biology 14:e1005944. https://doi.org/10.1371/journal.pcbi.1005944

42. Smith JR, Hayman GT, Wang S-J (2020) The Year of the Rat: The Rat Genome Database at 20: a multi-species knowledgebase and analysis platform. Nucleic Acids Research 48:. https://doi.org/10.1093/nar/gkz1041

43. Li L, Wu Z, Guarracino A (2025) Genetic modulation of protein expression in rat brain. iScience 28:112079. https://doi.org/10.1016/j.isci.2025.112079

Figure Captions

Fig. 1. Overview of the pangenome workflow described in this protocol. Starting from de novo genome assemblies (left), the PGGB pipeline performs all-to-all alignment (WFMASH), graph induction (SEQWISH), graph normalization (SMOOTHXG), and redundancy removal (GFAFFIX) to produce a pangenome graph in GFA format. Downstream analyses include variant calling with vg deconstruct, structural variant detection via a multi-method approach, read mapping with vg Giraffe, and phenome-wide association mapping (PheWAS) linking novel variants to phenotypes.

Fig. 2. Example ODGI visualizations of a pangenome graph. (A) 1D visualization (odgi viz): each horizontal row represents a genome path through the graph. Colored segments indicate graph node traversals; gaps correspond to sequence absent from that path. (B) 2D layout (odgi layout + odgi draw): the topological structure of the graph, where linear stretches appear as straight lines and variant sites (bubbles) produce branching patterns.